The Ultimate Beginner's Guide to **Claude Code**

Claude Code, built by Anthropic, is the most powerful **agentic coding tool** available today. It turns your terminal (or IDE) into a collaborative workspace where Claude isn't just a chatbot—it's an autonomous co-engineer that reads your entire codebase, plans solutions, writes code, runs tests, debugs errors, handles git, and even opens PRs—all through natural language.

Unlike traditional AI coding assistants that suggest snippets, Claude Code *acts*. It can iterate for hours unsupervised, making it feel like you've hired a senior engineer who never sleeps.

This guide takes you from zero to advanced, incorporating the best community-discovered hacks, workflows, and pro tips as of November 2025.

Level 1: Foundations: Get Running in 10 Minutes

1. Installation Options

Environment	Recommended Method	Command / Steps
macOS	Native installer (fastest, no Node required)	curl -fsSL https://claude.ai/install.sh | bash
Linux	Native binary	Same curl script as above
Windows	Use WSL2 + Ubuntu (best experience) or native PowerShell installer	In PowerShell: iwr https://claude.ai/install.ps1 -useb | iex
VS Code / Cursor / Windsurf	VS Code extension (beta but excellent)	Search "Claude Code" in VS Code Marketplace

After install, run claude in any project folder. It will prompt for your Anthropic API key (get one at console.anthropic.com).

Pro tip: Run claude doctor to verify everything is healthy.

2. Pricing & Model Selection (November 2025)

Tier	Monthly Cost	Best Models Included	Ideal For
Pro	$20	Claude Sonnet 4.5 (default)	Everyday coding, debugging, learning
Max	$100–$200+ (usage-based tiers)	Claude Opus 4.1 + Sonnet 4.5	Massive refactors, architecture, long sessions
Team/Enterprise	Custom	All models + higher limits	Production teams

Sonnet 4.5 is the sweet spot for agentic work—blazing fast and smarter than Opus on most coding benchmarks. Switch with /model inside a session.

3. The Single Most Important File: CLAUDE.md

This Markdown file is Claude Code's **persistent brain**. Every session automatically prepends its contents to context.

- Create it: Run /init → Claude scans your repo and generates a perfect starter.
- Keep it updated: Press # in the terminal and say "add this to CLAUDE.md" → auto-appended.
- Golden contents to include:
  - High-level architecture diagram (text or mermaid)
  - Run commands (npm run dev, docker compose up, etc.)
  - Coding conventions & style guide
  - Git workflow rules ("never commit to main", feature-branch pattern)
  - Important env variables or secrets handling
  - "Red lines" – things Claude must NEVER touch

Hack: Paste your company's entire internal wiki/snippet library here for god-tier context.

4. Core Commands & Shortcuts You'll Use 100x

Action	Command / Shortcut	Description
Start / resume session	claude or claude --resume	
Reference a file	@path/to/file.ext + Tab completion	Instantly adds file to context
Long instructions	@instructions.md	Keep prompts clean
Interrupt Claude	Esc (once = pause, twice = edit last prompt)	
Check token usage	/usage	Critical for staying under context limits
Compact conversation	/compact	Summarizes history when nearing 70-80% context
Hard reset	/clear	Start fresh (use sparingly)
Update CLAUDE.md	#	"Remember this forever" shortcut

Level 2: Workflow Mastery – Ship Features Like a 10x Engineer

The Gold Standard: Explore → Plan → Code → Commit (EPCC)

1. **Explore** – "Read these files and summarize the auth flow but DO NOT write code yet: @src/auth/"
2. **Plan** – Press Shift + Enter (or type /plan) → "Create a detailed step-by-step plan with file changes."
   - Edit the plan yourself before approving!
3. **Code** – Say "Execute the plan."
4. **Commit** – Claude writes perfect commit messages and can push + open PRs.

Prompting Hacks That 10x Output Quality

Keyword / Phrase	Effect
think hard / ultrathink	Forces deeper reasoning before acting
think step by step	Classic Chain-of-Thought – reduces hallucinations dramatically
write tests first	Triggers true TDD – Claude writes failing tests → implements → iterates
be extremely precise	Great for refactors
verify with a subagent	Spawns a second Claude to double-check

Context Engineering Mastery

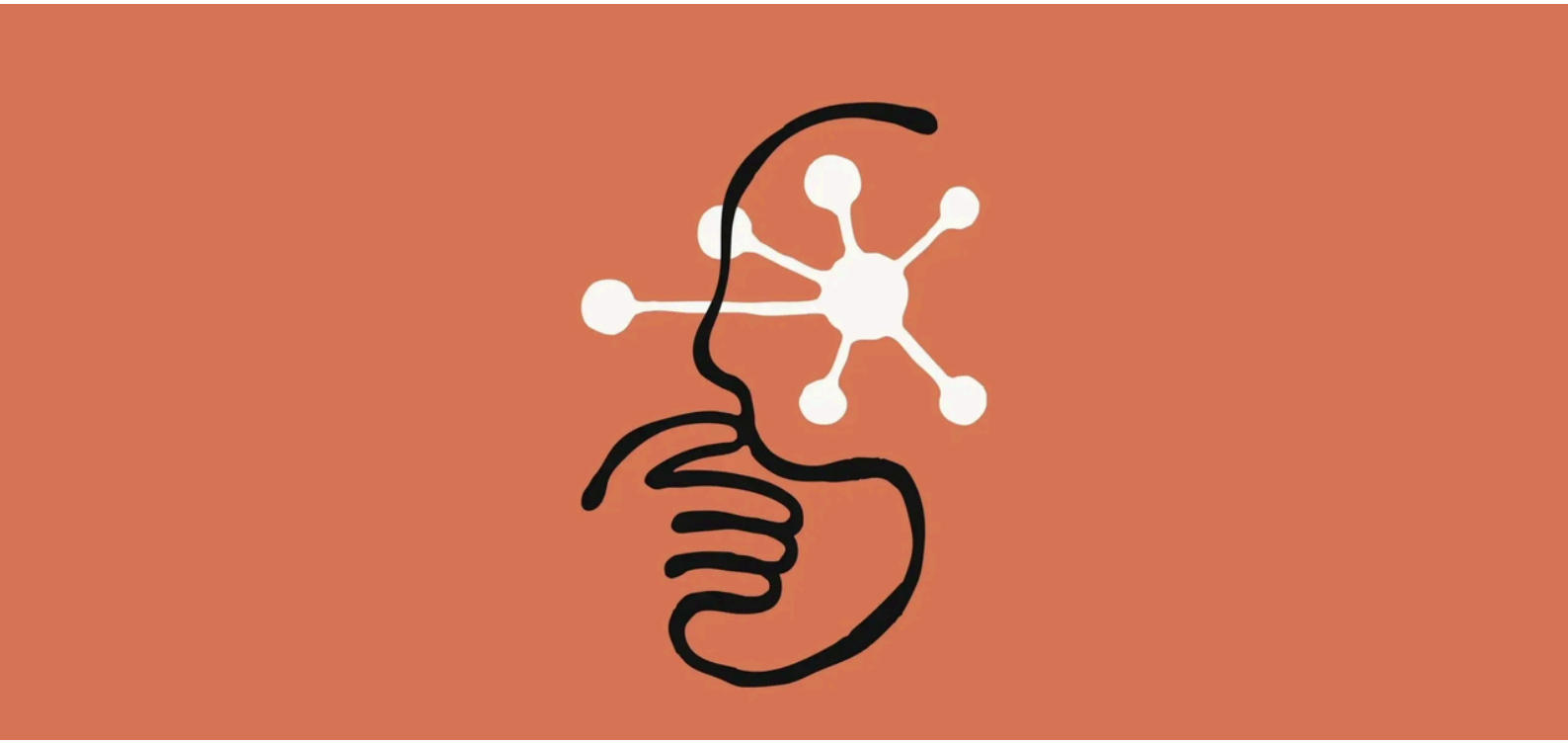
Context is your biggest bottleneck. Master these:

Technique	Why It's Magic	How
.claudeignore	Excludes node_modules, dist/, logs, etc. → saves 50%+ tokens	Create in project root, gitignore-style syntax
Workspace isolation	In monorepos, cd into the specific package first	Limits file discovery
Scratchpad files	External markdown files for long plans/checklists → survive /clear	plan.md, todo.md – tell Claude to read/write them
Manual @ references	Prevents Claude from reading 500 irrelevant files	Always @ the 3-5 most important files first

Quality & Debugging Superpowers

- **TDD on Steroids**: "Write comprehensive tests for X, then implement until all pass."

- **Screenshot → Fix**: Drag images into terminal (macOS/Linux) or paste URLs – Claude sees UI bugs.
- **Security Review**: /security-review or just ask – finds SQLi, XSS, auth flaws instantly.
- **Lint/Format Automation**: "Fix all eslint errors" → runs autonomously.



Level 3: Advanced & "Scary Good" Techniques

Custom Automation

Feature	What It Does	Setup Location
Custom Slash Commands	Turn multi-step workflows into /mycommand	.claude/commands/mycommand.md
Subagents	Delegate specialized tasks (e.g., DB expert, UI tester)	/agents create db-expert "...prompt"
Hooks	Run code before/after edits (lint, typecheck, etc.)	settings.local.json
Skills / SOPs	Reusable standard procedures	Markdown files Claude auto-invokes

Parallel & Multi-Agent Development

- Run 5+ terminal tabs with separate Claude instances on different features.
- Use Git Worktrees for true parallel branches without conflicts.
- "Spawn 3 subagents to explore different architectures, then pick the best."
- Verification hack: One Claude writes code → second separate instance reviews/tests.

Model Context Protocol (MCP) – Supercharge With External Tools

MCP connects Claude to the world:

- Playwright MCP → full browser automation & visual testing
- Database MCPs (Postgres, Mongo, Supabase)
- Figma → design-to-code
- Slack, Google Drive, Notion

Enable in settings.local.json → no more "I can't access that".

GitHub & CI Integration

- Tag @claude in issues/PRs → auto-fixes and submits PRs
- /install gh-actions → fully automated AI PR bot
- Claude resolves merge conflicts better than most seniors

Autonomy & Safety

Mode	Use Case	How to Enable
Auto-accept (session only)	Trusted repos, fast iteration	Alt + M or configure per-session
Full YOLO (--dangerously-skip-permissions)	Max speed (use sandbox/DevContainer!)	claude --dangerously-skip-permissions
Sandboxed mode	Safe unsupervised runs	Latest versions have OS-level sandboxing

Warning: Full autonomy is incredibly powerful but risky. Always use Dev Containers or git branches.

The Co-Engineer Mindset

Stop micromanaging lines of code. Start thinking like a CTO:

- Give Claude the **vision** and **constraints**
- Provide perfect context (CLAUDE.md, @files, tests)
- Let it handle the 90% grind work
- You verify architecture, tests, and product fit

Analogy: Claude Code is like hiring a brilliant but slightly eccentric senior engineer. Give them detailed specs, a style guide, and clear boundaries—and they'll ship faster than your whole team.

You now have everything the top 1% of Claude Code users know.

Go build something ridiculous.

Love & Automation,

Jack 🤖